Predictive Real Estate Pricing Platform

Project Documentation

Version 1.0 | October 2026

Stack: HTML, CSS, JavaScript, Python, Flask, SQLite

Table of Contents

1. Executive Summary 4

2. Problem Statement 4

3. Objectives 4

3.1 Scope 4

4. Target Users 4

5. Functional Requirements 4

6. Non-Functional Requirements 5

7. System Architecture 5

7.1 Components 6

7.2 Request Flow 6

8. Technology Stack 7

9. Data Design 7

9.1 Data Sources 7

9.2 Data Dictionary 8

9.3 Entity Relationships 8

9.4 Database Schema (SQLite) 9

9.5 Data Cleaning Rules 9

10. Machine Learning Methodology 9

10.1 Features 9

10.2 Models 9

10.3 Training Pipeline 10

10.4 Evaluation Metrics 10

10.5 Training Code Sketch 11

10.6 Price Range and Confidence 11

11. Verdict Logic 11

12. API Specification 12

12.1 POST /api/predict 12

12.2 GET /api/trend/ 12

12.3 Other Endpoints 13

12.4 Reference Implementation 13

13. User Interface Design 13

13.1 Pages 14

13.2 UX Guidelines 14

13.3 Visual Design System . . . 14
13.4 Desktop Screens . . . 15
13.5 Mobile Screens . . . 17
14. Project Structure . . . 18
15. Implementation Plan . . . 18
15.1 Milestones . . . 18
16. Testing Strategy . . . 19
17. Deployment . . . 19
18. Cost Estimate . . . 19
19. Risks and Mitigation . . . 20
20. Legal Disclaimer . . . 20
21. Future Enhancements . . . 20
22. Glossary . . . 21
23. Conclusion . . . 21

1. Executive Summary

The Predictive Real Estate Pricing Platform is an intelligent web application that estimates the fair market price of a property from its location, area, age, and other attributes, using a machine learning model trained on historical sales data. The platform compares the estimated price with the seller's asking price and tells the user whether the property is **overpriced**, **fairly priced**, or **a good opportunity**. It also visualizes how prices in the selected district changed over the last two years.

The system is built with a lightweight and widely used stack: **HTML, CSS, JavaScript, Python, Flask, and SQLite**, with scikit-learn (and optionally XGBoost) as the prediction engine.

2. Problem Statement

Property pricing in many markets is opaque and subjective:

- **Buyers** cannot easily tell whether an asking price is reasonable or inflated.
- **Real estate offices** often price properties using personal experience and intuition, which leads to inconsistent valuations between agents.
- **Investors** lack a simple, data-driven way to identify districts with rising prices or undervalued deals.

There is a need for a fast, transparent, and data-backed tool that gives an objective price reference.

3. Objectives

1. Predict the fair price of a property from its characteristics with an average error below 10%.
2. Compare the predicted price against the asking price and produce a clear verdict.
3. Display district-level price trends over the past 24 months (with an optional short-term forecast).
4. Show comparable past transactions to build user trust in the estimate.
5. Provide a simple, responsive interface usable by non-technical users.

3.1 Scope

In scope: price prediction, asking-price comparison, district trend charts, comparable sales table, prediction history storage, basic tooling for loading data.

Out of scope (v1): official or legal valuation, payment processing, user-to-user messaging, a property listing marketplace, native mobile apps.

4. Target Users

User group	Needs	Benefit from the platform
Real estate offices	Fast, consistent pricing	Number-backed valuations and higher credibility with clients
Buyers	Verify an offer is fair	Avoid overpaying and negotiate with evidence
Investors	Find growing districts and undervalued deals	Trend analysis and opportunity detection

5. Functional Requirements

The platform must satisfy the following functional requirements.

ID	Requirement	Priority
FR-1	The user can select a city and district.	High
FR-2	The user can enter area (m^2), property age, rooms, bathrooms, floor, and finishing level.	High
FR-3	The user can enter the seller's asking price.	High
FR-4	The system returns a predicted price and a price range.	High
FR-5	The system computes the difference (%) between asking and predicted price and returns a verdict.	High
FR-6	The system shows a line chart of the average price per m^2 in the district over the last 24 months.	High
FR-7	The system lists similar properties sold in the same district.	Medium
FR-8	The system stores each prediction request for analytics.	Medium
FR-9	The system displays a confidence indicator based on data availability in the district.	Medium
FR-10	The system displays a disclaimer that results are indicative, not an official valuation.	High

6. Non-Functional Requirements

- **Performance:** prediction response time under 2 seconds.
- **Accuracy:** mean absolute percentage error (MAPE) below 10% on the test set where data volume is sufficient.
- **Usability:** the form is completed in under one minute; the interface is responsive on mobile and desktop.
- **Reliability:** input validation and graceful error messages for all endpoints.
- **Maintainability:** modular code, a separate training script, and a versioned model file.
- **Security:** parameterized SQL queries, server-side validation, no sensitive personal data stored.
- **Portability:** runs locally with a single command and deploys to free hosting platforms.

7. System Architecture

The system follows a classic three-tier architecture: a browser-based presentation layer, a Flask application layer that exposes a REST API, and a data and model layer made of a serialized ML model and a SQLite database. Training is an offline step that reads from the database and produces the model file.

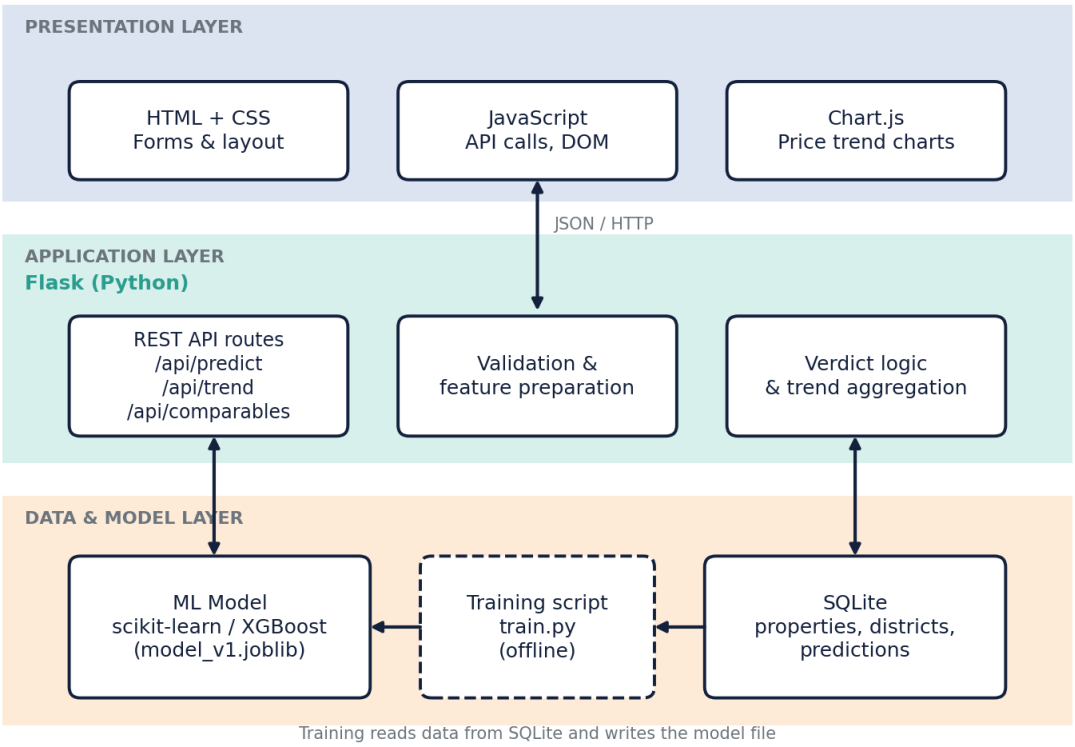


Figure 1 - System architecture (layers and components)

7.1 Components

- **Presentation layer:** HTML forms, CSS styling, JavaScript for API calls and Chart.js for charts.
- **Application layer:** Flask routes, validation logic, verdict calculation, trend aggregation.
- **Model layer:** a pre-trained regression model serialized with joblib and loaded once at server start.
- **Data layer:** SQLite stores historical transactions, district metadata, and logged predictions.

7.2 Request Flow

Figure 2 shows what happens when a user submits the form and the result page loads its trend chart.

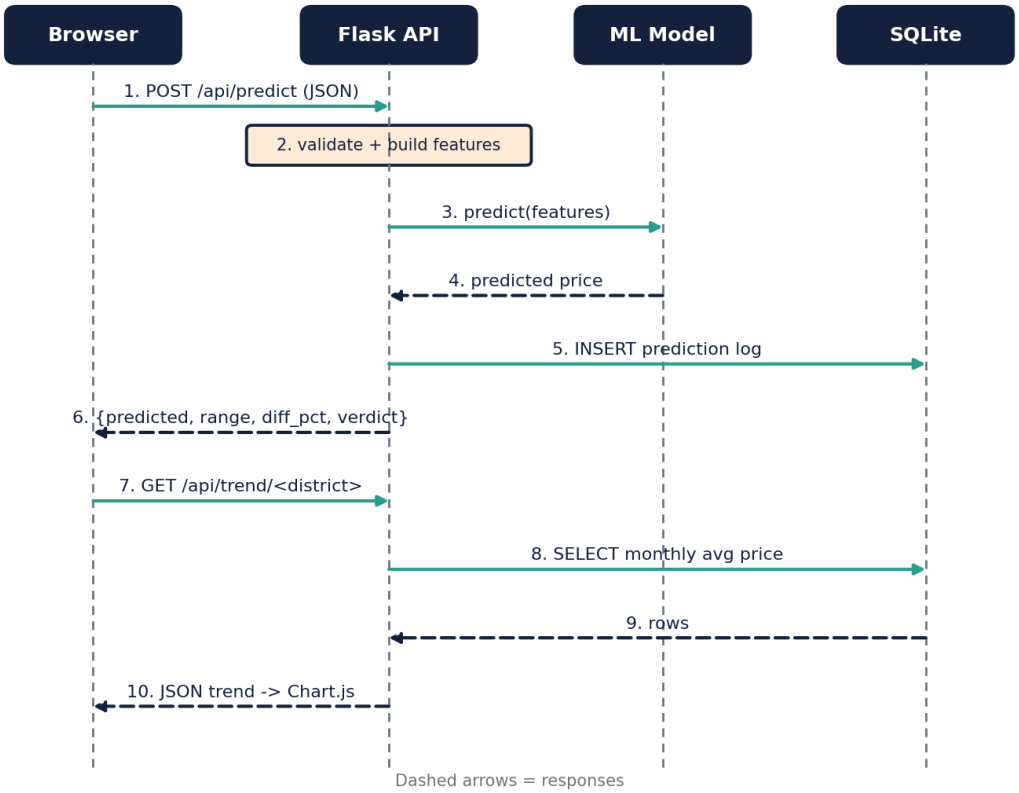


Figure 2 - Sequence diagram of a prediction request and trend loading

8. Technology Stack

Technology	Role
HTML5	Page structure and forms
CSS3	Styling and responsive layout
JavaScript (ES6)	Client logic, API calls, DOM updates
Chart.js	Price trend visualization
Python 3.10+	Backend logic, data processing, model training
Flask	Web framework and REST API
scikit-learn	Preprocessing, Linear Regression, Random Forest
XGBoost (optional)	Gradient boosting model
pandas / NumPy	Data cleaning and analysis
joblib	Model persistence
SQLite	Embedded relational database

9. Data Design

9.1 Data Sources

1. **Public datasets** (for example, Kaggle datasets on Egyptian real estate).

2. **Scraped listings** from real estate websites. The terms of service and robots.txt of each site must be reviewed before any scraping.
3. **Manually collected transactions** from partner real estate offices, which are the most accurate but slowest to gather.

Important: advertised prices are usually higher than the final sale prices. Where possible, prefer actual closed-sale data, or apply a documented adjustment factor.

9.2 Data Dictionary

Field	Type	Description
district	Categorical	District or neighborhood name
area_m2	Numeric	Property area in square meters
age_years	Numeric	Age of the building in years
rooms	Integer	Number of rooms
bathrooms	Integer	Number of bathrooms
floor	Integer	Floor number
finishing	Categorical	Finishing level (none, semi, full, luxury)
has_elevator	Boolean	Whether the building has an elevator (optional)
distance_services_km	Numeric	Distance to key services (optional)
price	Numeric	Sale price (target variable)
sale_date	Date	Date of the transaction

9.3 Entity Relationships

A district has many properties. Predictions are stand-alone log records that store the request as JSON together with the output.



Figure 3 - Entity-relationship diagram

9.4 Database Schema (SQLite)

```
CREATE TABLE districts (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    name      TEXT NOT NULL,
    city      TEXT NOT NULL,
    lat       REAL,
    lng       REAL,
    UNIQUE(name, city)
);

CREATE TABLE properties (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    district_id  INTEGER NOT NULL REFERENCES districts(id),
    area_m2      REAL    NOT NULL,
    age_years    INTEGER NOT NULL,
    rooms        INTEGER,
    bathrooms    INTEGER,
    floor        INTEGER,
    finishing    TEXT,
    price        REAL    NOT NULL,
    sale_date    DATE    NOT NULL
);

CREATE TABLE predictions (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    input_json       TEXT NOT NULL,
    predicted_price  REAL NOT NULL,
    asking_price     REAL NOT NULL,
    diff_pct         REAL NOT NULL,
    verdict          TEXT NOT NULL,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_properties_district_date ON properties(district_id, sale_date);
```

9.5 Data Cleaning Rules

- Remove exact duplicate records.
- Remove outliers using the IQR rule or percentile clipping on price per m^2.
- Drop records with missing price, area, or district.
- Impute missing numeric values (rooms, floor) with the district median; impute categorical values with the mode.
- Discard impossible values (area <= 0, negative age, price <= 0).
- Normalize district names (spelling variants, Arabic/English forms).

10. Machine Learning Methodology

10.1 Features

District, area, age, rooms, bathrooms, floor, finishing level, and optionally elevator availability and distance to services. The sale date (as a year-month index) is included so the model can learn market inflation over time.

10.2 Models

Model	Purpose
Linear Regression	Simple baseline for comparison
Random Forest Regressor	Strong non-linear model, robust to noise
Gradient Boosting / XGBoost	Typically the best accuracy on tabular property data

10.3 Training Pipeline

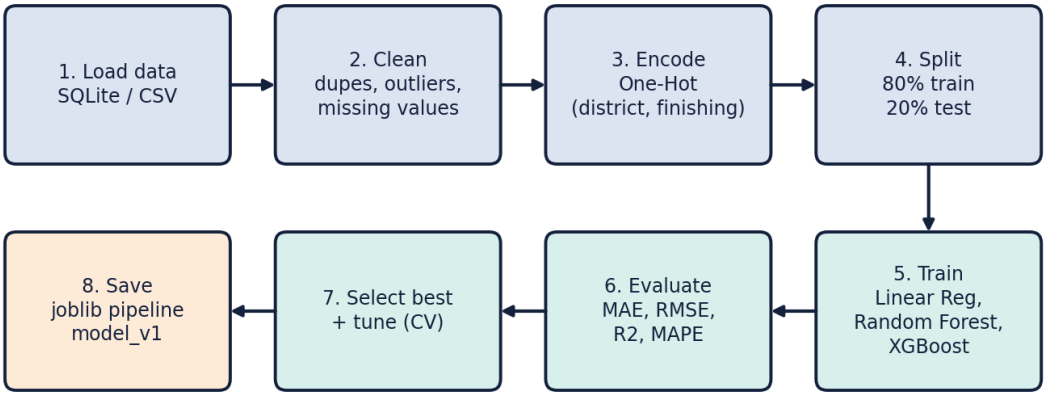


Figure 4 - Model training and export pipeline

1. **Load** data from SQLite or CSV into a pandas DataFrame.
2. **Clean** the data using the rules in section 9.5.
3. **Encode** categorical variables (One-Hot Encoding for district and finishing).
4. **Split** the dataset: 80% training and 20% testing (use a time-aware split if the sale date is a feature).
5. **Train** each candidate model and tune hyperparameters with cross-validation (for example, 5-fold GridSearchCV).
6. **Evaluate** using MAE, RMSE, R^2, and MAPE.
7. **Select** the best model and save it with joblib together with the fitted encoder as a single pipeline.
8. **Version** the model file (for example, model_v1.joblib) and record its metrics.

10.4 Evaluation Metrics

Metric	Meaning	Target
MAE	Average absolute error in currency units	As low as possible
RMSE	Penalizes large errors	As low as possible
R^2	Share of variance explained	Above 0.80
MAPE	Average percentage error	Below 10%

10.5 Training Code Sketch

```
import pandas as pd, joblib
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

df = pd.read_csv("properties_clean.csv")
X = df.drop(columns=["price"])
y = df["price"]

cat = ["district", "finishing"]
num = ["area_m2", "age_years", "rooms", "bathrooms", "floor"]

pre = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
    ("num", "passthrough", num),
])

model = Pipeline([
    ("pre", pre),
    ("rf", RandomForestRegressor(n_estimators=300, random_state=42)),
])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
model.fit(X_tr, y_tr)

pred = model.predict(X_te)
print("MAE:", mean_absolute_error(y_te, pred))
print("R2:", r2_score(y_te, pred))

joblib.dump(model, "model_v1.joblib")
```

10.6 Price Range and Confidence

- **Price range:** use the spread of predictions across the trees of a Random Forest (for example, the 10th and 90th percentiles), or apply the model's historical error percentage around the point estimate.
- **Confidence level:** derived from the number of training samples in the selected district (fewer than 30 = Low, 30 to 100 = Medium, more than 100 = High). The level is shown to the user.

11. Verdict Logic

The difference between the asking price and the predicted price is calculated as:

```
diff_pct = (asking_price - predicted_price) / predicted_price * 100
```

Condition	Verdict	Badge
diff_pct > +10%	Overpriced	Red
-10% <= diff_pct <= +10%	Fair price	Yellow
diff_pct < -10%	Good opportunity	Green

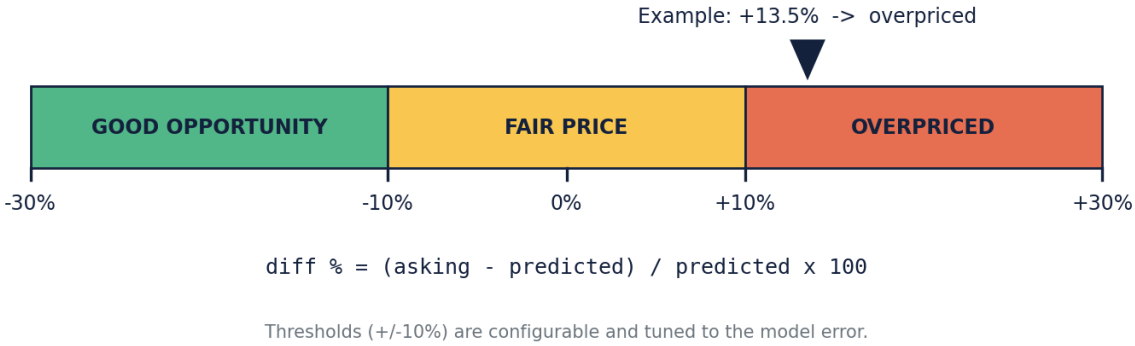


Figure 5 - Verdict zones on the price-difference scale

The +/-10% thresholds are configurable and should be tuned according to the model's observed error.

12. API Specification

All endpoints exchange JSON.

12.1 POST /api/predict

Request

```
{
  "city": "Mansoura",
  "district": "El Gomhoria",
  "area_m2": 140,
  "age_years": 8,
  "rooms": 3,
  "bathrooms": 2,
  "floor": 4,
  "finishing": "full",
  "asking_price": 2100000
}
```

Response

```
{
  "predicted": 1850000,
  "range": [1750000, 1950000],
  "diff_pct": 13.5,
  "verdict": "overpriced",
  "confidence": "medium"
}
```

Errors: 400 for missing or invalid fields, 404 for an unknown district, 500 for unexpected server errors.

12.2 GET /api/trend/<district>

Returns the monthly average price per m^2 for the last 24 months.

```
{
  "district": "El Gomhoria",
  "months": ["2024-11", "2024-12", "..."],
  "avg_price_m2": [11200, 11350, "..."]
}
```

12.3 Other Endpoints

Endpoint	Description
GET /api/comparables?district=...&area=...	Up to 10 recently sold properties in the same district with similar area.
GET /api/districts?city=...	List of supported districts for the dropdown.

12.4 Reference Implementation

```
from flask import Flask, request, jsonify
import joblib, pandas as pd

app = Flask(__name__)
model = joblib.load("model_v1.joblib")

def verdict(diff):
    if diff > 10:  return "overpriced"
    if diff < -10: return "opportunity"
    return "fair"

@app.route("/api/predict", methods=["POST"])
def predict():
    data = request.get_json()
    required = ["district", "area_m2", "age_years", "asking_price"]
    if not all(k in data for k in required):
        return jsonify(error="Missing fields"), 400

    X = pd.DataFrame([data]).drop(columns=["asking_price"])
    price = float(model.predict(X)[0])
    diff = (data["asking_price"] - price) / price * 100

    return jsonify(
        predicted=round(price),
        diff_pct=round(diff, 1),
        verdict=verdict(diff),
    )

@app.route("/api/trend/<district>")
def trend(district):
    return jsonify(get_monthly_avg_price(district, months=24))

if __name__ == "__main__":
    app.run(debug=True)
```

13. User Interface Design

The interface consists of two main screens: an input form and a result view that combines the verdict, the trend chart, and comparable sales.

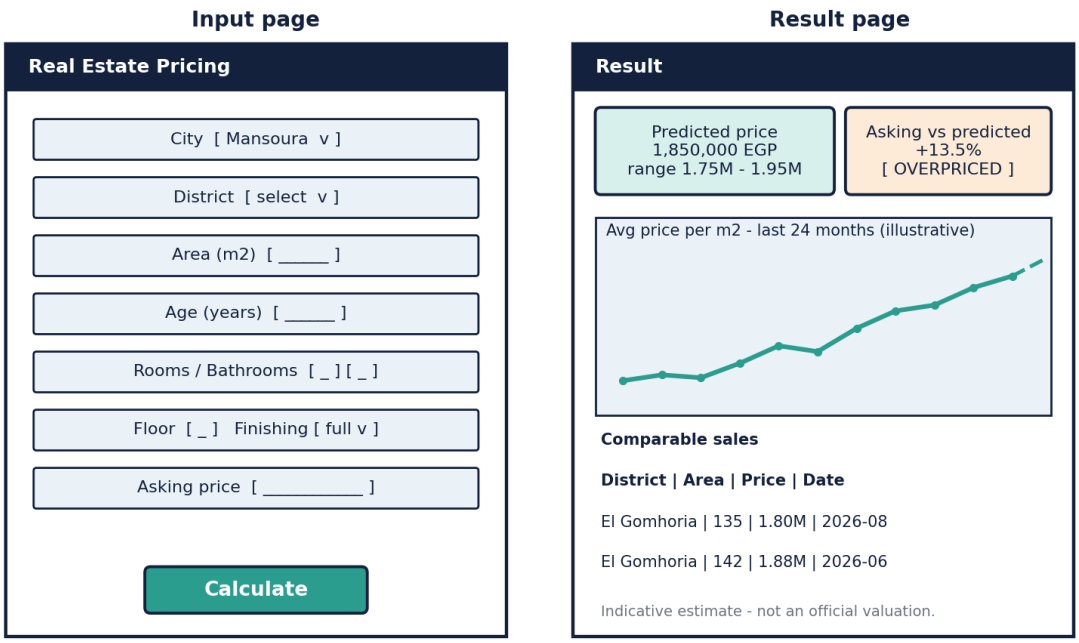


Figure 6 - Low-fidelity wireframes of the input page and the result page

13.1 Pages

1. **Home / Input page:** city and district selectors (optionally a map picker); fields for area, age, rooms, bathrooms, floor, and finishing; optional extras; the asking price; and a Calculate button.
2. **Result page:** predicted price and range, difference from the asking price with a colored verdict badge, a confidence indicator, a 24-month trend chart, a comparable sales table, and the disclaimer.
3. **History page (optional):** previous predictions.

13.2 UX Guidelines

- Right-to-left (RTL) and Arabic labels supported alongside English.
- Inline validation with clear error messages.
- Loading indicator while waiting for the prediction.
- Color-blind friendly badges (use icons and text, not color alone).
- Responsive layout for phones and tablets.

13.3 Visual Design System

The interface follows a restrained, professional look inspired by modern fintech and property dashboards: one typeface, one accent color, hairline borders, and subtle shadows. The design is Arabic-first (right-to-left) and mirrors cleanly for English.

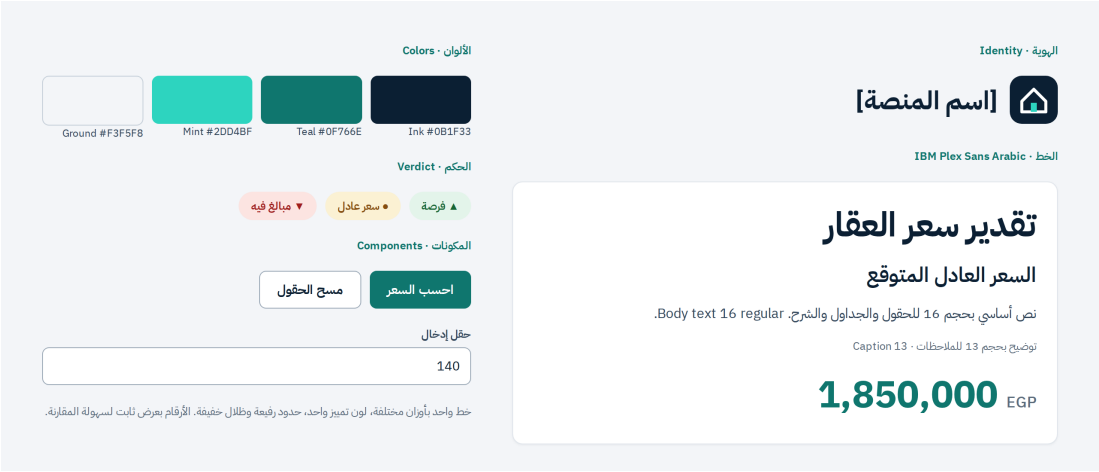


Figure 7 - Style guide: identity, typography, colors, verdict badges, and base components

Token	Value	Usage
Typeface	IBM Plex Sans Arabic (400, 500, 600, 700)	All text; tabular figures for prices and tables
Ink	#0B1F33	Primary text, logo, header marks
Teal	#0F766E	Primary actions, links, predicted price
Mint	#2DD4BF	Logo accent only
Ground / Card	#F3F5F8 / #FFFFFF (border #E1E6EC)	Page background and surfaces
Opportunity	Text #166534 on #E3F4EA	Asking price more than 10% below prediction
Fair price	Text #854D0E on #FBF1D3	Within +/-10% of prediction
Overpriced	Text #9F1D1D on #FCE4E1	Asking price more than 10% above prediction
Type scale	44/700 display, 28-30/700 title, 16/600 section, 14-16/400 body, 12-13 caption	Hierarchy for numbers, headings, and notes
Spacing and shape	8px grid; 8px radius for controls, 12px for cards	Consistent rhythm across screens

- **Accessibility:** verdict badges combine an icon, a color, and text so meaning never depends on color alone; text colors meet contrast requirements on their backgrounds; mobile controls are at least 48px tall.
- **Placeholders:** the platform name, the map area, and all chart and table values are illustrative and are replaced with real data during implementation.

13.4 Desktop Screens

Input screen. A single card groups the form into three numbered sections (location, property specifications, asking price). A side column holds an optional map for pinning the location and a short summary of what the user will receive.

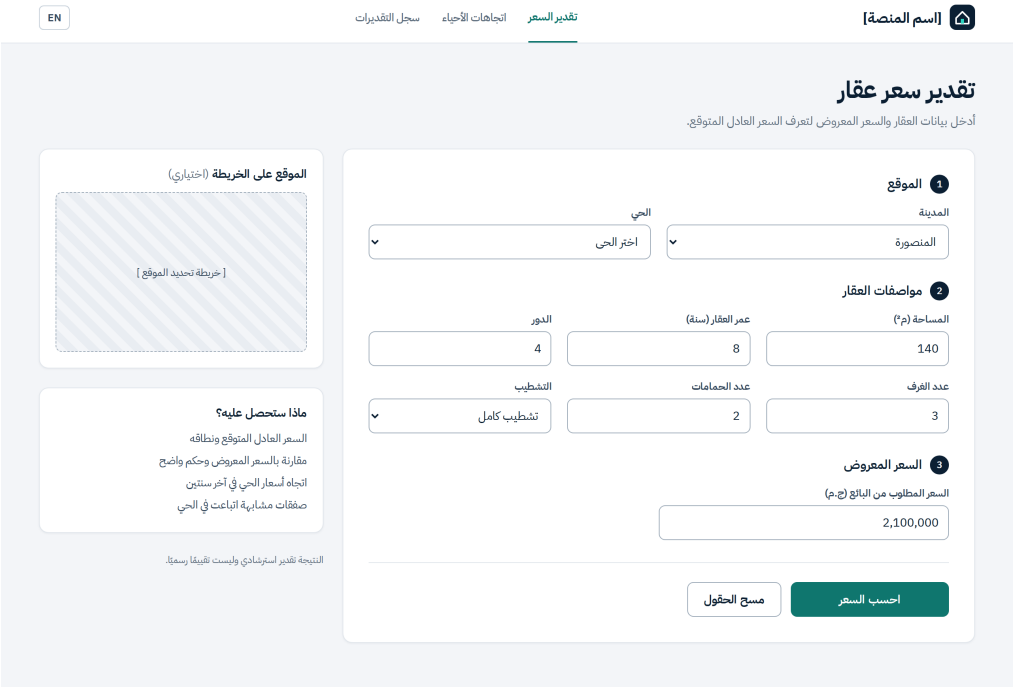


Figure 8 - Desktop input screen

Result screen. The page leads with the two most important answers: the fair predicted price with its range, and the verdict. The range bar shows where the asking price falls relative to the predicted range. Below, four KPI cards give the price per m^2 (predicted and asking), the difference in currency, and the confidence level, followed by the 24-month trend chart and a list of comparable sales.



Figure 9 - Desktop result screen

13.5 Mobile Screens

On phones the layout becomes a single column. Small fields pair up in two columns, the map becomes an optional button, and the result screen reorders its cards so the verdict appears first, followed by the predicted price and range bar, a 2x2 grid of KPIs, the trend chart, comparable sales, and full-width action buttons.

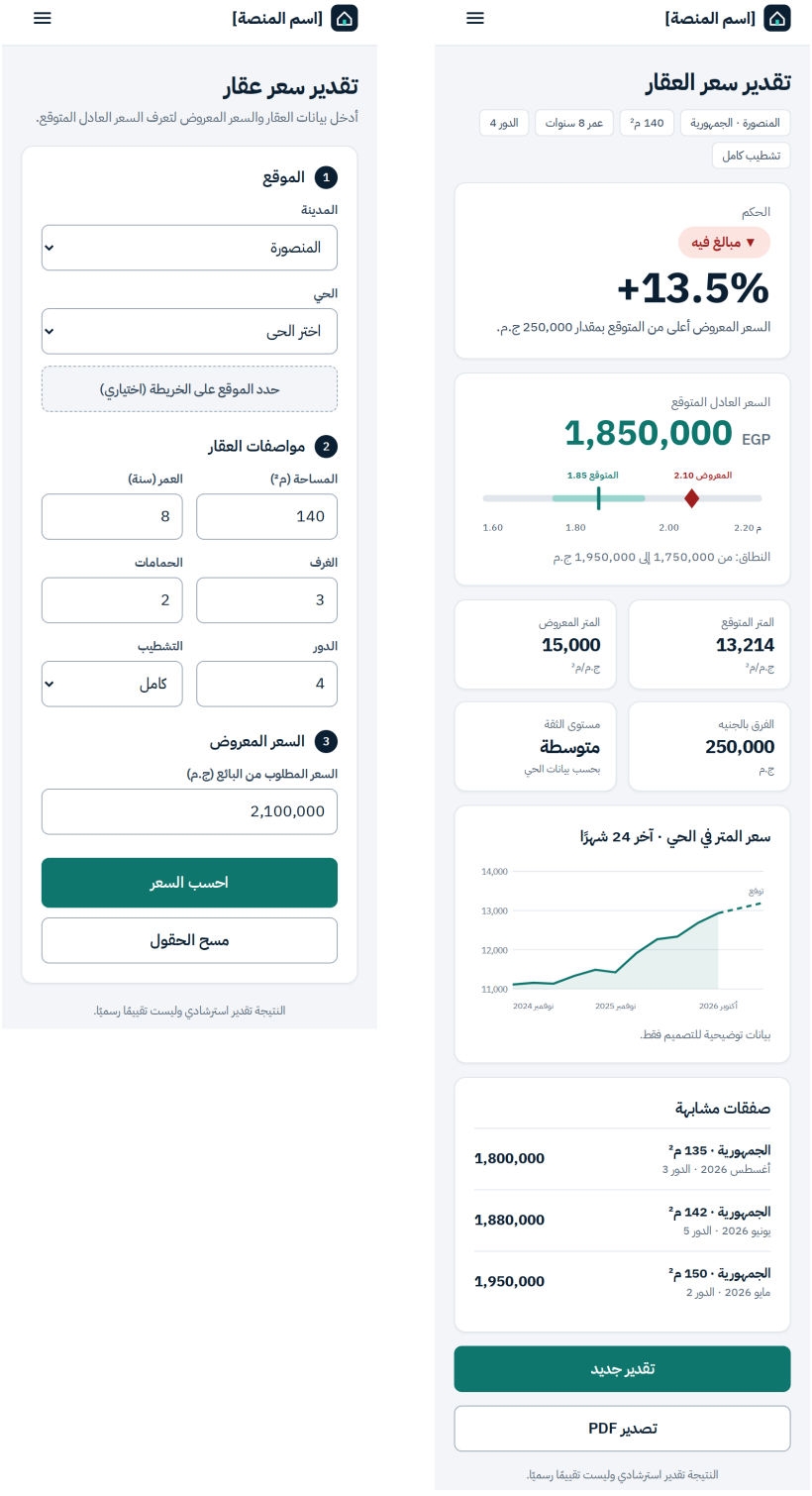


Figure 10 - Mobile input screen (left) and result screen (right), 390px wide

14. Project Structure

```
real-estate-pricing/
|-- app.py                  # Flask application
|-- requirements.txt
|-- data/
|   |-- raw/                # original datasets
|   `-- properties_clean.csv
|-- ml/
|   |-- train.py            # training and evaluation script
|   `-- model_v1.joblib
|-- db/
|   |-- schema.sql
|   `-- real_estate.db
|-- templates/
|   |-- index.html
|   `-- result.html
|-- static/
|   |-- css/style.css
|   `-- js/app.js
`-- tests/
    |-- test_api.py
    `-- test_model.py
```

15. Implementation Plan

Phase	Tasks	Duration
1. Data	Collect, clean, and load the dataset into SQLite	1-2 weeks
2. Modeling	Train, tune, and evaluate models; export the best one	1 week
3. Backend	Build the Flask API and database access layer	1 week
4. Frontend	Build the forms, result view, and Chart.js graphs	1 week
5. Integration and release	Connect, test, and deploy	3-5 days

Total estimated duration: 4 to 6 weeks for one developer.

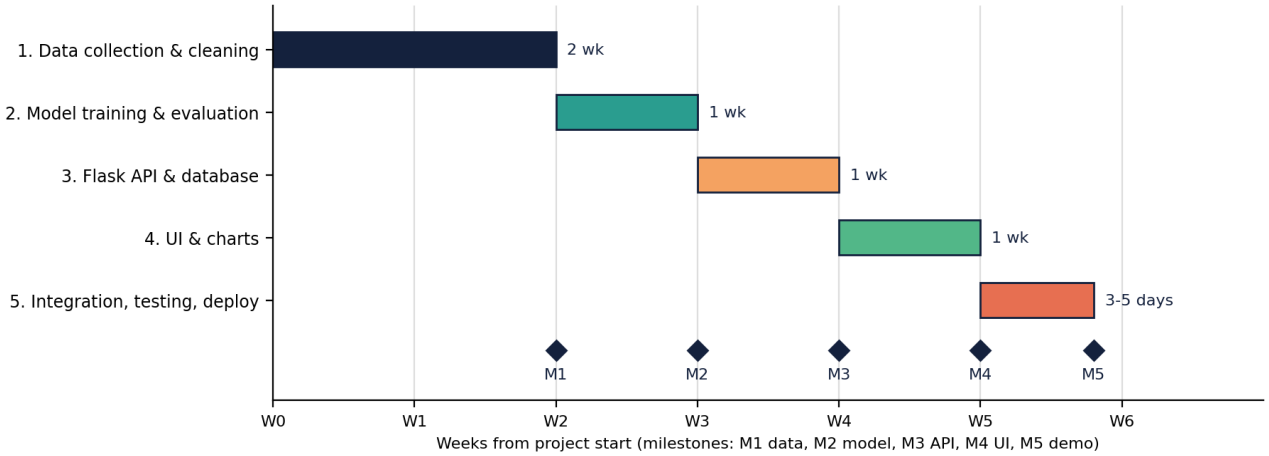


Figure 11 - Project timeline and milestones

15.1 Milestones

- **M1:** clean dataset and database ready.
- **M2:** model reaches target accuracy.
- **M3:** API returns predictions end to end.
- **M4:** complete UI connected to the API.

- **M5:** deployed demo with documentation.

16. Testing Strategy

Level	What is tested	Tools
Unit	Verdict function, data cleaning, feature preparation	pytest
Model	Metrics on held-out test set, behavior on edge cases	scikit-learn metrics
API	Valid and invalid requests, status codes, response shape	pytest + Flask test client
UI	Form validation, chart rendering, responsive layout	Manual checks / browser dev tools
Acceptance	Realistic sample properties checked against expert estimates	Manual review with a real estate agent

Sample test cases

- Missing district returns 400.
- Asking price 20% above prediction returns overpriced.
- Asking price 15% below prediction returns opportunity.
- Unknown district returns a clear error message.
- Very small and very large areas are handled without crashing.

17. Deployment

- **Local:** pip install -r requirements.txt, then python app.py.
- **Hosting options:** Render or PythonAnywhere (both have free tiers suitable for a prototype).
- **Production notes:** use Gunicorn behind a web server, disable Flask debug mode, store configuration in environment variables, and back up the SQLite file regularly. For higher traffic, migrate to PostgreSQL.

18. Cost Estimate

The project is designed to be nearly free to build and run as a prototype.

Item	Estimated cost
Development tools (Python, Flask, VS Code, libraries)	Free
Datasets (public data)	Free
Hosting (free tier)	Free
Domain name (optional)	Approximately 10-15 USD per year
Paid data or API access (optional)	Variable

The main real cost is the developer's time and, if needed, the effort of collecting high-quality transaction data.

19. Risks and Mitigation

Risk	Impact	Mitigation
Poor or biased data (asking vs. sale prices)	Inaccurate predictions	Prefer closed-sale data, document assumptions, apply adjustment factors
Few records for some districts	Low accuracy there	Show a confidence level; fall back to broader area averages
Rapid market changes and inflation	Model becomes outdated	Include sale date as a feature and retrain on a schedule
Legal or ethical concerns about scraping	Possible violations	Respect terms of service and robots.txt; prefer open datasets
Users treating the output as an official valuation	Misuse and liability	Display a clear disclaimer on every result
Model overfitting	Poor real-world performance	Cross-validation, held-out test data, regularization

20. Legal Disclaimer

The price shown by this platform is an **indicative estimate** generated by a statistical model. It is **not** an official or certified property valuation and must not be used as the sole basis for any financial or legal decision. Users should consult a licensed appraiser before buying, selling, or investing.

21. Future Enhancements

- Heatmap of district prices on an interactive map.
- Rental yield and payback-period calculator for investors.
- Numerical confidence score for each prediction.
- Email alerts when a below-market opportunity appears.
- Dedicated accounts for real estate offices with PDF report export.
- Model explainability (feature importance or SHAP) so users see why a price was estimated.
- Automated monthly data refresh and model retraining.
- Mobile app version.

22. Glossary

Term	Definition
MAE	Mean Absolute Error, the average size of prediction errors
RMSE	Root Mean Squared Error, an error metric that penalizes large mistakes
R^2	Coefficient of determination, how much variance the model explains
MAPE	Mean Absolute Percentage Error
One-Hot Encoding	Converting categories into binary columns
Random Forest	Ensemble of decision trees that averages their predictions
Gradient Boosting	Ensemble method that builds trees sequentially to correct earlier errors
REST API	Interface that lets the frontend communicate with the backend over HTTP

23. Conclusion

This project delivers a practical, data-driven tool that brings transparency to property pricing. With a compact technology stack, a clear architecture, and a staged plan, it can be built as a graduation project or a startup prototype, then extended with maps, investor tools, and professional features for real estate offices.